UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA

MANEJO E IMPLEMENTACIÓN DE ARCHIVOS 1

LABORATORIO — PROYECTO 1

ExtreamFS

Nombre: Yury Urbano Santos Chagil
Carnet: 202113318
Semestre: 1S2026
Fecha: 18 de Marzo de 2026

INTRODUCCIÓN

El presente manual técnico describe la arquitectura interna, los módulos principales y el flujo de funcionamiento del proyecto ExtreamFS, desarrollado como una aplicación web para la simulación de un sistema de archivos EXT2. El sistema permite administrar discos, particiones, formateo, usuarios, grupos, carpetas, archivos y reportes estructurales, integrando un backend en C++ con un frontend en Angular.

El objetivo de este manual es documentar el diseño interno del sistema, explicar cómo interactúan sus componentes y servir como referencia técnica para comprender el procesamiento de comandos y la generación de reportes.

OBJETIVO DEL SISTEMA

El sistema fue desarrollado para simular operaciones fundamentales de un sistema de archivos EXT2 sobre discos virtuales creados en archivos binarios. Entre sus funciones principales se encuentran:

- creación y eliminación de discos
- administración de particiones primarias, extendidas y lógicas
- montaje de particiones
- formateo en EXT2
- administración de usuarios y grupos
- creación de carpetas y archivos
- lectura de archivos
- generación de reportes estructurales
- ejecución de comandos desde una aplicación web

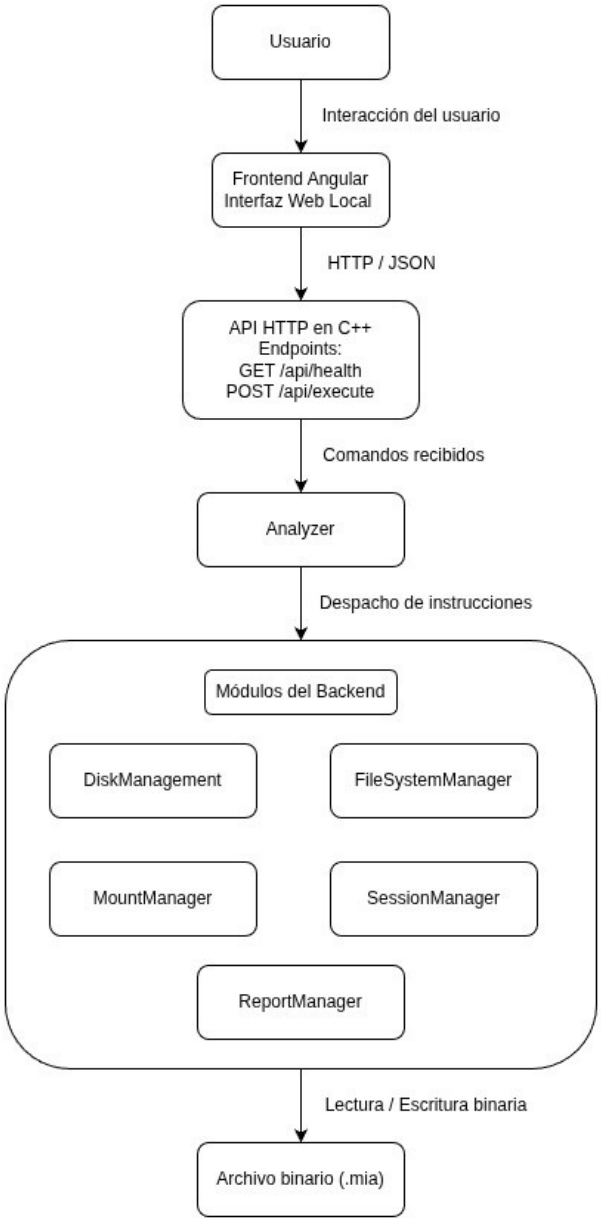
ARQUITECTURA DEL SISTEMA

La arquitectura del proyecto está dividida en dos capas principales:

- **Frontend**
- **Backend**

Esta separación permite mantener desacoplada la interfaz gráfica de la lógica del sistema de archivos. El frontend se encarga de la interacción con el usuario y el backend procesa la lógica de negocio, manipula el disco binario y genera la salida correspondiente.

Arquitectura general



Descripción por capas

Frontend

El frontend fue implementado con Angular y cumple la función de interfaz web del sistema. Sus responsabilidades principales son:

- recibir comandos manuales desde un área de texto
- cargar archivos de script `.smia`
- enviar el contenido al backend mediante peticiones HTTP
- mostrar la salida generada por el sistema en una consola
- mostrar el estado de conexión del backend

En la versión desarrollada, la interfaz contiene una sección para ingreso manual de comandos, una sección para visualizar archivos `.smia` cargados y una consola de salida donde se muestran comentarios, errores y resultados de ejecución.

Backend

El backend fue implementado en C++, cumpliendo con la tecnología exigida por el proyecto. Esta capa centraliza la lógica principal del sistema y se encarga de:

- interpretar comandos
- validar parámetros
- manipular discos y particiones
- administrar el sistema de archivos EXT2
- manejar sesiones, usuarios y grupos
- generar reportes con Graphviz
- exponer servicios HTTP para el frontend

El backend trabaja directamente sobre archivos binarios `.mia`, donde se almacena toda la estructura simulada del sistema de archivos.

Componentes internos del backend

La lógica del backend está organizada en módulos especializados.

Analyzer

La clase `Analyzer` es el núcleo del sistema. Su función es recibir líneas de texto, interpretarlas y redirigir cada comando al módulo correspondiente.

Responsabilidades principales:

- identificar el comando ingresado
- separar y normalizar parámetros
- validar sintaxis
- invocar la operación correspondiente
- devolver mensajes de éxito o error

Además del modo interactivo en consola, el proyecto incorporó un método de ejecución por script, utilizado por la API para procesar el contenido enviado desde el frontend.

DiskManagement

Este módulo administra todo lo relacionado con discos y particiones.

Responsabilidades:

- crear discos virtuales (`mkdisk`)
- eliminar discos (`rmdisk`)
- crear particiones (`fdisk`)
- validar restricciones de teoría de particiones
- calcular tamaños y posiciones dentro del archivo binario

Aquí se implementa la lógica para particiones:

- primarias
- extendidas
- lógicas

También se manipula la información del MBR y de los EBR cuando existen particiones lógicas dentro de una extendida.

MountManager

Este componente administra el montaje de particiones.

Responsabilidades:

- registrar particiones montadas
- asignar IDs de montaje
- evitar montajes duplicados
- listar particiones montadas con `mounted`

La estructura de montaje se mantiene en memoria mientras el backend está activo. Por ello, el orden de montaje influye directamente en IDs como `181A`, `182A`, `181B`, etc.

FileSystemManager

Este módulo implementa la lógica principal del sistema de archivos EXT2.

Responsabilidades:

- formatear particiones (`mkfs`)
- administrar carpetas y archivos (`mkdir`, `mkfile`)
- leer archivos (`cat`)
- manipular inodos y bloques
- escribir y leer estructuras EXT2 dentro del archivo binario

Aquí se encuentran las operaciones relacionadas con:

- super bloque
- bitmap de inodos
- bitmap de bloques
- tabla de inodos
- bloques de carpetas
- bloques de archivos

SessionManager

Este módulo administra la autenticación y sesión actual del sistema.

Responsabilidades:

- iniciar sesión (`login`)
- cerrar sesión (`logout`)
- validar si existe sesión activa
- mantener el usuario actual
- controlar permisos para operaciones restringidas

También se apoya en el archivo especial `users.txt`, donde se registran grupos y usuarios.

ReportManager

Este componente genera los reportes visuales y de texto solicitados por el proyecto.

Responsabilidades:

- construir el contenido DOT para Graphviz
- generar archivos `.jpg`, `.png` o `.txt`
- representar estructuras internas del sistema de archivos
- producir reportes como:
    - `mbr`
    - `disk`
    - `inode`
    - `block`
    - `bm_inode`
    - `bm_block`
    - `sb`
    - `file`
    - `ls`
    - `tree`

Este módulo utiliza Graphviz, como requiere el proyecto, para convertir estructuras internas del sistema en representaciones gráficas.

HttpServer

Para conectar el backend con el frontend, se agregó una capa HTTP en C++.

Responsabilidades:

- exponer endpoints para el frontend
- verificar el estado del backend
- recibir comandos o scripts desde Angular
- devolver la salida como JSON

Endpoints principales:

- `GET /api/health`
- `POST /api/execute`

Esta capa permite reutilizar toda la lógica del sistema sin modificar la esencia del backend CLI.

Arquitectura del frontend

El frontend fue diseñado como una interfaz web local y está compuesto por componentes y servicios principales.

app.component

Es el componente principal de la interfaz.

Responsabilidades:

- mantener el contenido del área de comandos
- leer el archivo `.smia` cargado
- ejecutar comandos manuales
- ejecutar el archivo cargado
- mostrar la salida en consola
- actualizar el estado del backend

backend.service

Este servicio encapsula la comunicación HTTP con el backend.

Responsabilidades:

- consultar el estado del backend
- enviar bloques de comandos al endpoint de ejecución
- recibir respuestas en formato JSON

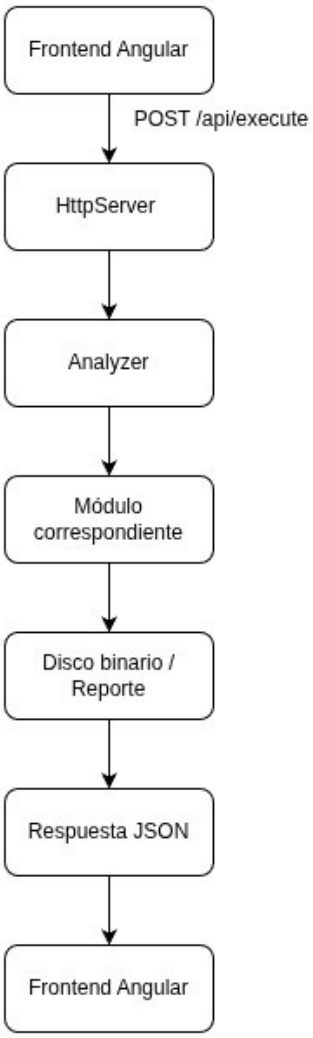
Gracias a este servicio, la interfaz no contiene lógica de negocio; solo consume la API.

Flujo de comunicación entre frontend y backend

El flujo de ejecución del sistema ocurre de la siguiente manera:

1. El usuario abre la aplicación web.
2. El frontend consulta el endpoint `GET /api/health`.
3. Si el backend responde, la interfaz muestra “Backend: Conectado”.
4. El usuario escribe comandos o carga un archivo `.smia`.
5. El frontend envía el contenido al endpoint `POST /api/execute`.
6. El servidor HTTP del backend recibe la petición.
7. El contenido es enviado al `Analyzer`.
8. El `Analyzer` procesa línea por línea.
9. Según el comando, se delega la ejecución al módulo correspondiente.
10. La salida se captura y se devuelve al frontend.
11. El frontend muestra la respuesta en la consola.

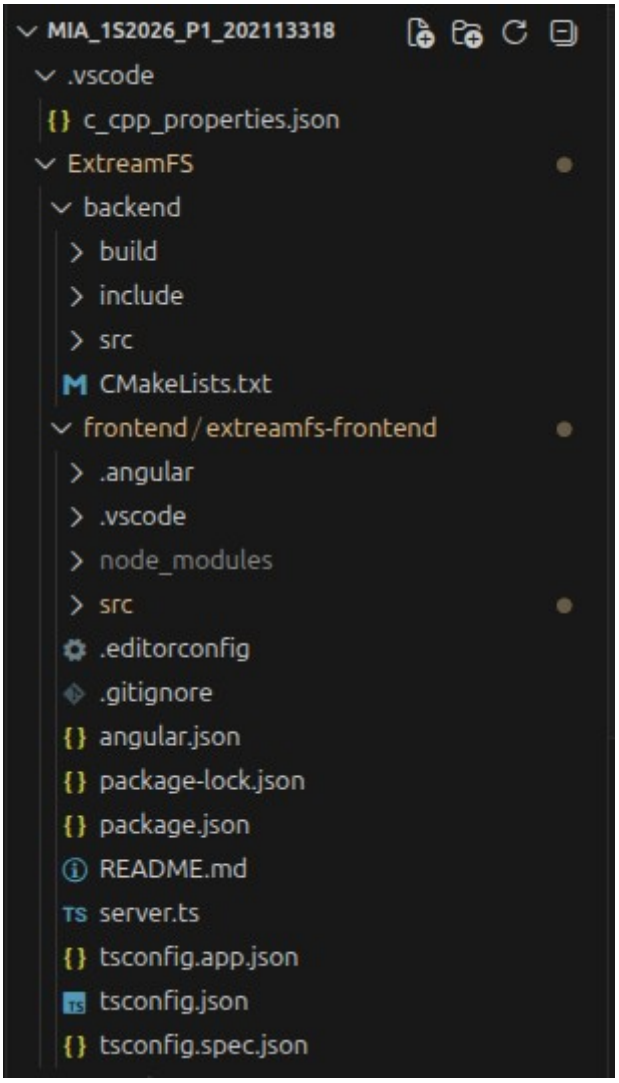
Diagrama de flujo de ejecución



Organización física del proyecto

La implementación del proyecto quedó organizada físicamente en dos carpetas principales: `backend` y `frontend`.

Estructura general



Esta organización facilita el mantenimiento del proyecto y separa claramente la lógica del sistema de la interfaz.

Ventajas de la arquitectura implementada

La arquitectura adoptada ofrece varias ventajas técnicas:

- separación clara entre interfaz y lógica de negocio
- reutilización del mismo backend tanto para modo CLI como para modo web
- facilidad de mantenimiento por módulos
- escalabilidad para agregar nuevos comandos o endpoints
- menor acoplamiento entre componentes
- mejor claridad en la depuración de errores

Consideraciones técnicas

El backend mantiene en memoria cierta información de ejecución, especialmente:

- particiones montadas
- sesión activa

Por esta razón, para corridas limpias con IDs de montaje esperados, se recomienda reiniciar el backend antes de ejecutar el script oficial.

Asimismo, la lógica del sistema fue diseñada para trabajar directamente sobre archivos binarios, simulando el comportamiento de estructuras EXT2 de forma persistente dentro del archivo `.mia`.

Estructuras de datos

El sistema ExtreamFS trabaja directamente sobre archivos binarios con extensión `.mia`, los cuales representan discos virtuales. Dentro de estos archivos se almacenan distintas estructuras que permiten simular el comportamiento de un sistema de archivos EXT2, así como las estructuras necesarias para la administración de discos y particiones.

Las estructuras más importantes implementadas en el proyecto son:

- MBR
- particiones primarias y extendidas
- EBR
- SuperBloque
- bitmap de inodos
- bitmap de bloques
- inodos
- bloques de carpetas
- bloques de archivos

Cada una de estas estructuras cumple una función específica dentro del sistema y se organiza físicamente dentro del archivo binario del disco.

MBR

El MBR o Master Boot Record es la primera estructura que se escribe al momento de crear un disco con el comando `mkdisk`. Se almacena al inicio del archivo binario y contiene la información general del disco, junto con el arreglo de particiones principales.

Su propósito es servir como encabezado principal del disco y registrar la información necesaria para identificar:

- tamaño total del disco
- fecha de creación
- firma del disco
- ajuste general
- información de hasta cuatro particiones primarias o extendidas

Función dentro del sistema

El MBR es utilizado principalmente en:

- creación de discos
- creación de particiones
- lectura de metadatos generales del disco
- generación de reportes `mbr` y `disk`

Cuando el sistema ejecuta comandos como `fdisk`, primero lee el MBR para determinar:

- si todavía existe espacio disponible
- cuántas particiones ya están definidas
- si ya existe una partición extendida
- en qué posición puede colocarse una nueva partición

Campos principales del MBR

De manera conceptual, el MBR contiene:

- `mbr_tamano`
- `mbr_fecha_creacion`
- `mbr_disk_signature`
- `disk_fit`
- arreglo de 4 particiones

Organización dentro del archivo

El MBR se encuentra al inicio del archivo `.mia`, ocupando la primera porción del disco. Después de esta estructura comienzan las regiones donde pueden almacenarse las particiones.

Estructura de partición

Cada partición almacenada en el MBR contiene información como:

- estado
- tipo
- ajuste
- posición inicial
- tamaño
- nombre

Estas entradas permiten que el sistema identifique si una partición está:

- activa o vacía
- definida como primaria o extendida
- ubicada correctamente dentro del disco
- asociada a un nombre específico

Tipos de partición

El sistema maneja tres tipos de partición:

- **Primaria**
- **Extendida**
- **Lógica**

Las particiones primarias y extendidas se registran directamente en el MBR.
Las particiones lógicas no se almacenan directamente en el MBR, sino dentro de la partición extendida mediante estructuras EBR.

EBR

El EBR o Extended Boot Record es la estructura utilizada para administrar particiones lógicas dentro de una partición extendida.

Cuando se crea una partición extendida, esta funciona como un contenedor. Dentro de dicho contenedor se van encadenando estructuras EBR, una por cada partición lógica.

Función dentro del sistema

El EBR permite:

- registrar particiones lógicas
- enlazar una partición lógica con la siguiente
- recorrer secuencialmente todas las particiones lógicas dentro de la extendida
- validar espacio disponible en la región extendida

Información manejada por un EBR

De manera conceptual, un EBR incluye:

- estado
- ajuste
- posición inicial
- tamaño
- puntero o referencia al siguiente EBR
- nombre de la partición lógica

Organización dentro del archivo

El primer EBR se escribe al inicio de la región extendida.
Cuando se crean nuevas particiones lógicas, el sistema:

- localiza el último EBR válido
- calcula el siguiente espacio libre
- escribe un nuevo EBR
- actualiza el encadenamiento

Importancia técnica

Esta estructura es fundamental para que el sistema respete la teoría clásica de particiones, donde:

- solo puede existir una partición extendida
- las particiones lógicas viven dentro de esa extendida
- cada lógica se administra a través de EBRs enlazados

SuperBloque

El SuperBloque es la estructura principal del sistema de archivos EXT2 dentro de una partición formateada.

Se escribe al ejecutar `mkfs` y contiene toda la información de control necesaria para administrar la partición como sistema de archivos.

Función dentro del sistema

El SuperBloque se utiliza para:

- identificar el tipo de sistema de archivos
- conocer la cantidad de inodos y bloques
- conocer cuántos están libres
- ubicar el inicio de bitmaps, tabla de inodos y bloques
- registrar fechas de montaje y desmontaje
- servir como punto de referencia para todas las demás estructuras EXT2

Información principal del SuperBloque

Entre sus campos más importantes se encuentran:

- tipo de sistema de archivos
- número total de inodos
- número total de bloques
- número de inodos libres
- número de bloques libres
- fecha de montaje
- fecha de desmontaje
- contador de montajes
- tamaño del inodo
- tamaño del bloque
- primer inodo libre
- primer bloque libre
- inicio del bitmap de inodos
- inicio del bitmap de bloques
- inicio de la tabla de inodos
- inicio de la tabla de bloques

Organización dentro del archivo

Dentro de una partición formateada, el SuperBloque se coloca al inicio del área de la partición.
A partir de él se determinan las posiciones relativas del resto de estructuras.

Bitmap de inodos

El bitmap de inodos es una estructura auxiliar que permite conocer qué inodos están ocupados y cuáles se encuentran libres.

Función dentro del sistema

Se utiliza para:

- localizar rápidamente el siguiente inodo libre
- marcar inodos como ocupados o liberados
- actualizar el estado del sistema al crear archivos o carpetas

Representación lógica

Cada posición del bitmap representa un inodo:

- `0` indica libre
- `1` indica ocupado

Organización dentro del archivo

Se ubica después del SuperBloque, en la posición indicada por el propio SuperBloque.

Bitmap de bloques

El bitmap de bloques cumple la misma función que el bitmap de inodos, pero aplicado a los bloques de datos.

Función dentro del sistema

Permite:

- saber qué bloques están disponibles
- marcar bloques al crear carpetas o archivos
- llevar control sobre el espacio utilizado en la partición

Representación lógica

Cada posición del bitmap representa un bloque:

- `0` libre
- `1` ocupado

Organización dentro del archivo

Se ubica después del bitmap de inodos, según la posición calculada al formatear la partición.

Inodos

Los inodos son estructuras fundamentales del sistema de archivos EXT2.
Cada archivo o carpeta del sistema se representa mediante un inodo.

Un inodo no almacena directamente el contenido del archivo, sino la información de control y los apuntadores a bloques.

Función dentro del sistema

Los inodos permiten almacenar:

- identificador del usuario propietario
- identificador del grupo
- tamaño del archivo o carpeta
- fechas de acceso, creación y modificación
- tipo de objeto
- permisos
- arreglo de apuntadores a bloques

Tipos de inodo

El sistema maneja principalmente dos tipos:

- inodo de carpeta
- inodo de archivo

Estructura lógica

Cada inodo posee un arreglo de apuntadores.
Estos apuntadores pueden referenciar:

- bloques de carpetas
- bloques de archivos

En la simulación desarrollada, estos apuntadores permiten modelar el crecimiento del sistema y la relación entre directorios y archivos.

Organización dentro del archivo

La tabla de inodos se encuentra después de los bitmaps.
Cada inodo se almacena de forma consecutiva dentro de esa tabla.

Bloques

Los bloques son las estructuras donde se almacena la información real del sistema de archivos.
Dependiendo del tipo de inodo, los bloques pueden representar contenido de directorio o contenido de archivo.

Bloque de carpeta

Un bloque de carpeta almacena entradas de directorio.
Cada entrada enlaza un nombre con un número de inodo.

Función dentro del sistema

Permite representar la estructura jerárquica de carpetas y archivos.

Por ejemplo, una carpeta puede contener entradas como:

- `.`
- `..`
- `users.txt`
- `home`

o cualquier otro archivo o subcarpeta existente dentro de ella.

Información manejada

Cada entrada de carpeta registra:

- nombre del elemento
- índice o número del inodo asociado

Uso práctico

Cuando el sistema ejecuta comandos como:

- `mkdir`
- `mkfile`
- `cat`
- `rep tree`
- `rep ls`

recorre bloques de carpeta para localizar rutas, nombres y relaciones entre elementos.

Bloque de archivo

Un bloque de archivo almacena el contenido de texto de un archivo.

Función dentro del sistema

Se utiliza para guardar:

- contenido generado por tamaño
- contenido proveniente de archivos externos
- contenido interno del sistema, como `users.txt`

Uso práctico

Este bloque es fundamental en operaciones como:

- `mkfile`
- `cat`
- `rep file`
- `rep block`

Cuando se crea un archivo, el sistema reserva uno o más bloques de archivo y escribe allí el contenido correspondiente.

Relación entre inodos y bloques

La relación entre estas estructuras funciona de la siguiente manera:

- un archivo o carpeta tiene un inodo
- el inodo contiene apuntadores
- los apuntadores referencian bloques
- los bloques contienen datos reales

Para una carpeta:

- el inodo apunta a bloques de carpeta
- los bloques de carpeta enlazan nombres con inodos hijos

Para un archivo:

- el inodo apunta a bloques de archivo
- los bloques de archivo contienen el texto almacenado

Esta relación permite construir la estructura jerárquica del sistema y también facilita la generación de reportes como `tree`, `inode`, `block`, `file` y `ls`.

Archivo especial `users.txt`

Dentro del sistema de archivos existe un archivo especial llamado `users.txt`, utilizado para registrar:

- grupos
- usuarios
- eliminaciones lógicas

Función dentro del sistema

Este archivo es la base para operaciones de:

- `login`
- `mkgrp`
- `rmgrp`
- `mkusr`
- `rmusr`
- `chgrp`

Su contenido se organiza como texto estructurado, donde cada línea representa una entidad del sistema.

Importancia técnica

Aunque `users.txt` es un archivo lógico dentro del sistema de archivos, su contenido también termina almacenado mediante:

- un inodo
- uno o más bloques de archivo

Esto demuestra que incluso la información administrativa del sistema respeta la misma estructura general de EXT2.

Relación entre estructuras de partición y estructuras EXT2

El sistema trabaja en dos niveles estructurales:

Nivel de disco y particiones

En este nivel se encuentran:

- MBR
- entradas de partición
- EBR

Estas estructuras administran el disco y la distribución del espacio.

Nivel de sistema de archivos

Dentro de una partición formateada se encuentran:

- SuperBloque
- bitmaps
- inodos
- bloques

Estas estructuras administran la información interna del sistema de archivos EXT2.

Integración entre ambos niveles

El flujo general funciona así:

- se crea un disco
- se crea una partición
- se monta la partición
- se formatea la partición
- se inicializan estructuras EXT2 dentro de esa partición

Esto significa que las estructuras de partición permiten ubicar el espacio, mientras que las estructuras EXT2 permiten organizar y administrar el contenido dentro de ese espacio.

Uso de las estructuras en los reportes

Las estructuras implementadas no solo se utilizan para operación interna, sino también para la generación de reportes.

Reporte MBR

Muestra la información del MBR y de las particiones.

Reporte DISK

Representa gráficamente la distribución del disco y sus regiones libres u ocupadas.

Reporte INODE

Muestra los inodos utilizados y sus campos.

Reporte BLOCK

Muestra los bloques de datos del sistema.

Reporte bm_inode

Representa el bitmap de inodos.

Reporte bm_block

Representa el bitmap de bloques.

Reporte TREE

Muestra la relación jerárquica entre inodos y bloques.

Reporte SB

Presenta el contenido del SuperBloque.

Reporte FILE

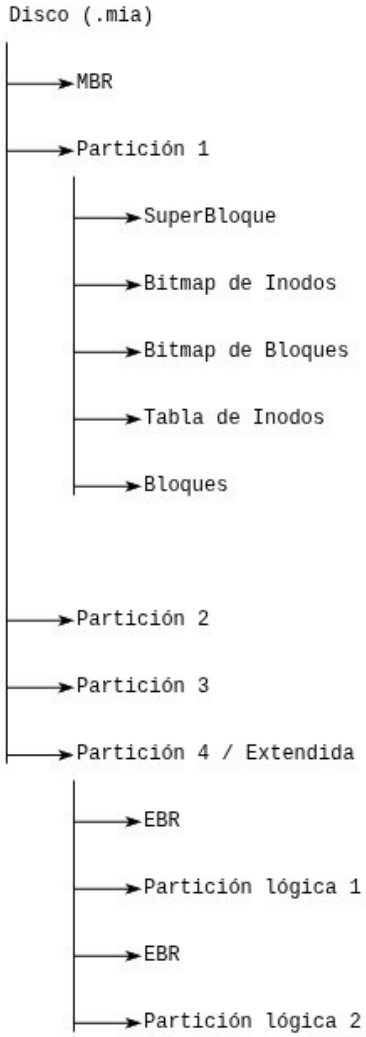
Muestra el contenido de un archivo específico.

Reporte LS

Muestra el contenido de una carpeta con información adicional.

Organización general dentro del archivo binario

De forma resumida, la organización del disco y del sistema de archivos dentro del archivo `.mia` puede representarse así:



Importancia de estas estructuras en el proyecto

La correcta implementación de estas estructuras permite que el sistema:

- respete la teoría de particiones
- simule EXT2 de forma coherente
- administre archivos y carpetas
- mantenga usuarios y grupos
- genere reportes consistentes
- procese comandos reales sobre una representación binaria organizada

Comandos Implementados

El sistema **ExtreamFS** implementa un conjunto de comandos orientados a la administración de discos, particiones, sistema de archivos, usuarios, grupos, carpetas, archivos y reportes. Cada comando es recibido por el `Analyzer`, validado sintácticamente y luego enviado al módulo correspondiente del backend.

En esta sección se describe:

- propósito de cada comando
- parámetros principales
- ejemplo de uso
- efecto sobre las estructuras internas del sistema

Comandos de administración de discos y particiones

MKDISK

El comando `mkdisk` crea un archivo binario que representa un disco virtual.

Propósito

Permite generar un disco nuevo con un tamaño específico, una unidad de medida y un tipo de ajuste. Este disco será el contenedor principal donde luego se crearán particiones y sistemas de archivos.

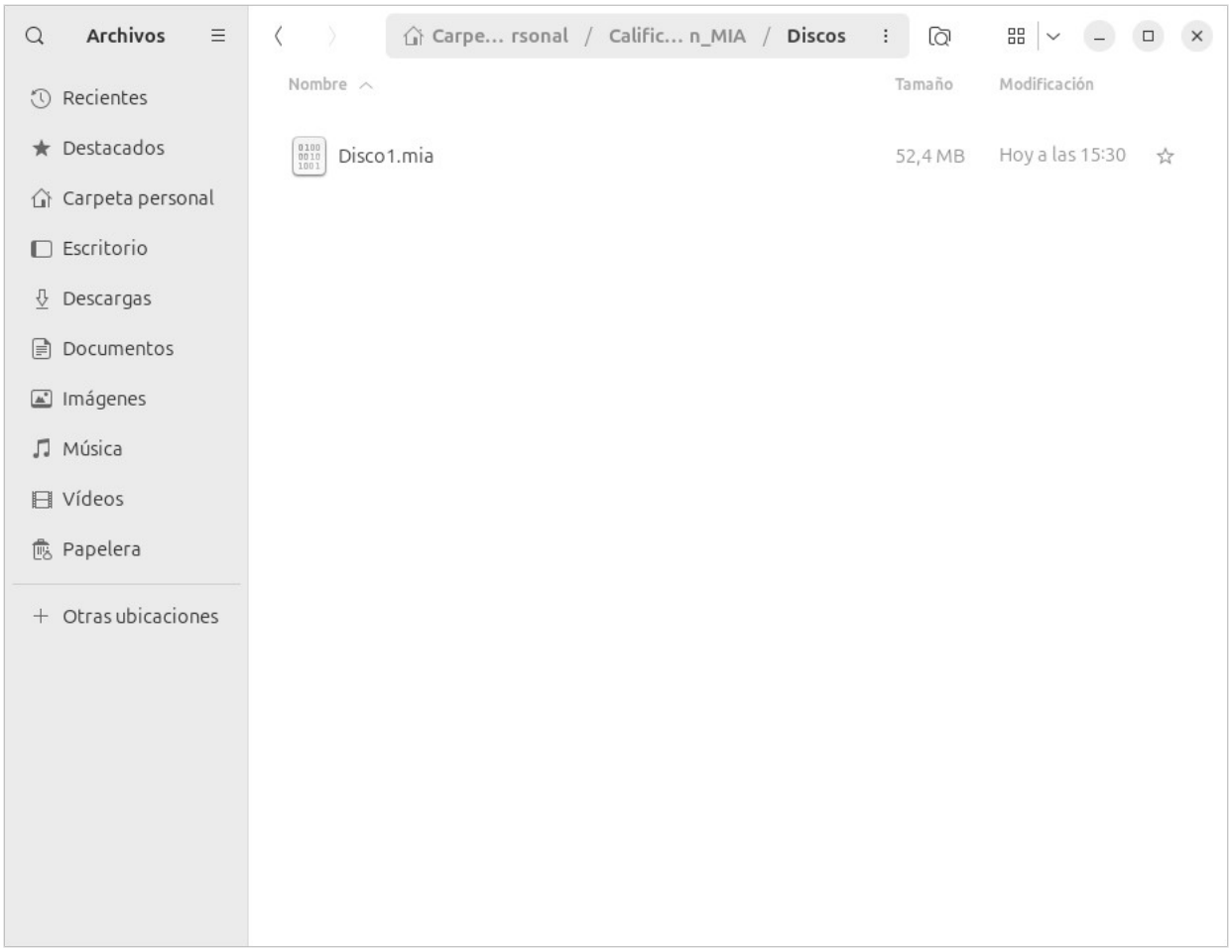
Parámetros principales

- `-size=` tamaño del disco
- `-unit=` unidad del tamaño (`b`, `k`, `m`)
- `-fit=` tipo de ajuste (`bf`, `ff`, `wf`)
- `-path=` ruta donde se creará el archivo `.mia`

Ejemplo

Entrada de comandos
fdisk -type=E -unit=K -name=Part51 -size=5120 -path=/home/yurysantos/Calificacion_MIA/Discos/Disco5.mia -fit=BF
Ejecutar
Limpiar

Salida / Consola
Consola lista.
[INFO] Ejecutando comandos manuales...
[OK] Disco creado OK: /home/yurysantos/Calificacion_MIA/Discos/Disco1.mia (52428800 bytes)


Archivos
Carpe... rsonal / Calific... n_MIA / Discos
Recientes
Destacados
Carpeta personal
Escritorio
Descargas
Documentos
Imágenes
Música
Vídeos
Papelera
Otras ubicaciones
Nombre
Tamaño
Modificación
Disco1.mia
52,4 MB
Hoy a las 15:30

Efecto en las estructuras internas

Al ejecutarse:

- se crea el archivo binario del disco
- se reserva el espacio total solicitado
- se escribe el **MBR** al inicio del archivo
- se inicializan las entradas de partición vacías
- se registra la fecha de creación, firma y ajuste del disco

RMDISK

El comando `rmdisk` elimina un disco previamente creado.

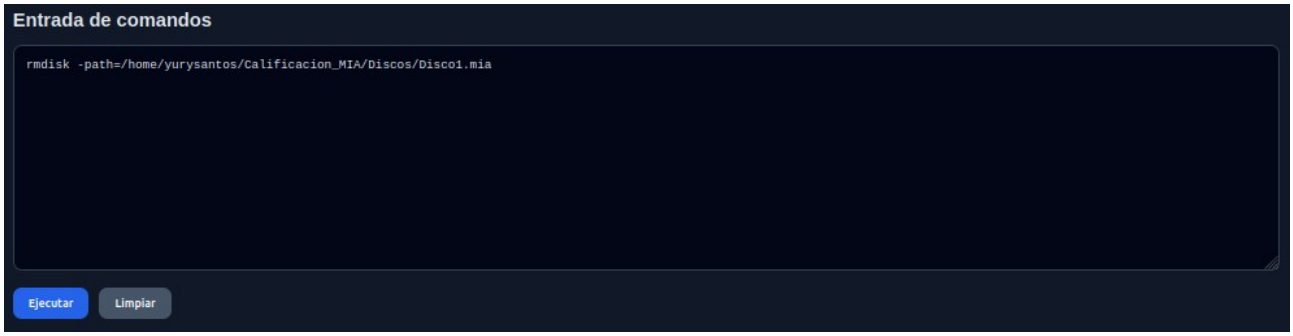
Propósito

Permite borrar el archivo binario que representa el disco virtual.

Parámetros principales

- `-path=` ruta del disco a eliminar

Ejemplo



FDISK

El comando `fdisk` crea particiones dentro de un disco existente.

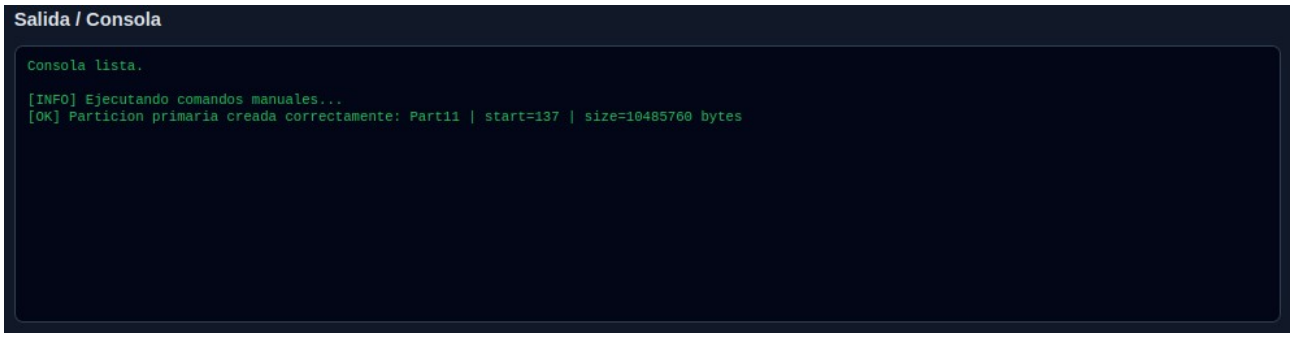
Propósito

Permite definir particiones primarias, extendidas o lógicas dentro del disco virtual, respetando las restricciones de teoría de particiones.

Parámetros principales

- `-size=` tamaño de la partición
- `-unit=` unidad (`b`, `k`, `m`)
- `-path=` ruta del disco
- `-name=` nombre de la partición
- `-type=` tipo (`p`, `e`, `l`)
- `-fit=` ajuste de la partición

Ejemplo de partición primaria



Ejemplo de partición extendida

Entrada de comandos

```
fdisk -type=E -unit=K -name=Part51 -size=5120 -path=/home/yurysantos/Calificacion_MIA/Discos/Disco5.mia -fit=BF
```

Ejecutar Limpiar

Salida / Consola

```
Consola lista.

[INFO] Ejecutando comandos manuales...
[OK] Particion extendida creada correctamente: Part51 | start=137 | size=5242880 bytes
```

Ejemplo de partición lógica

Entrada de comandos

```
fdisk -type=L -unit=K -name=Part52 -size=1024 -path=/home/yurysantos/Calificacion_MIA/Discos/Disco5.mia -fit=BF
```

Ejecutar Limpiar

Salida / Consola

```
Consola lista.

[INFO] Ejecutando comandos manuales...
[OK] Particion logica creada correctamente: Part52 | start=167 | size=1048576 bytes
```

Efecto en las estructuras internas

Según el tipo de partición:

Partición primaria

- se actualiza una de las cuatro entradas del MBR
- se define inicio, tamaño, nombre, ajuste y estado

Partición extendida

- también ocupa una entrada del MBR
- reserva una región que servirá de contenedor para particiones lógicas
- dentro de ella se escriben EBRs

Partición lógica

- se crea dentro de la región extendida
- se escribe un nuevo **EBR**
- se enlaza con EBRs anteriores si existen

Validaciones importantes

El comando valida:

- existencia del disco
- espacio disponible
- máximo de cuatro particiones primarias/extendidas
- existencia única de una partición extendida
- coherencia del tamaño y posición
- nombres duplicados de partición

MOUNT

El comando `mount` registra una partición como montada en memoria.

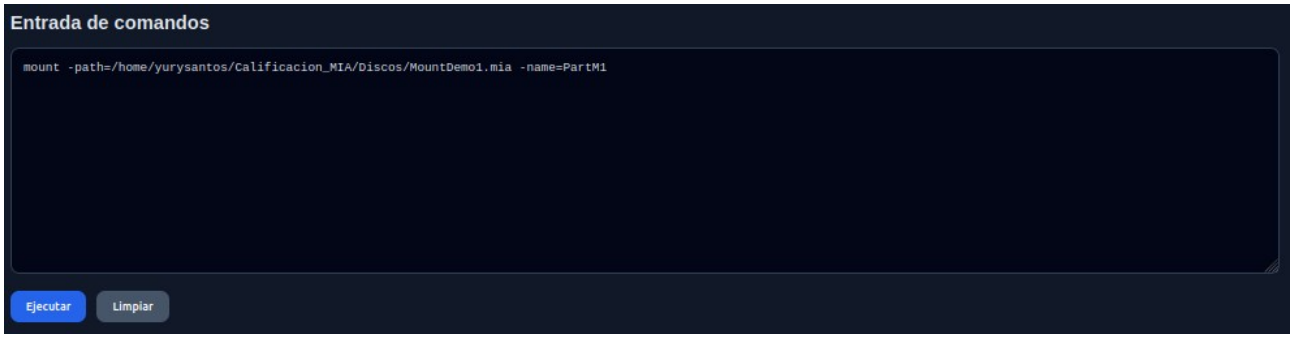
Propósito

Permite asociar una partición con un ID lógico, el cual será utilizado posteriormente por comandos como `mkfs`, `login` y `rep`.

Parámetros principales

- `-path=` ruta del disco
- `-name=` nombre de la partición

Ejemplo



Efecto en las estructuras internas

Al ejecutarse:

- el sistema localiza la partición dentro del disco
- registra la partición en la tabla de montajes en memoria
- genera un ID de montaje único, por ejemplo `181A`
- deja disponible esa partición para operaciones posteriores

Validaciones importantes

- no permite montar una partición inexistente
- no permite montar dos veces la misma partición
- verifica coherencia entre disco y nombre de partición

MOUNTED

El comando `mounted` lista las particiones montadas actualmente.

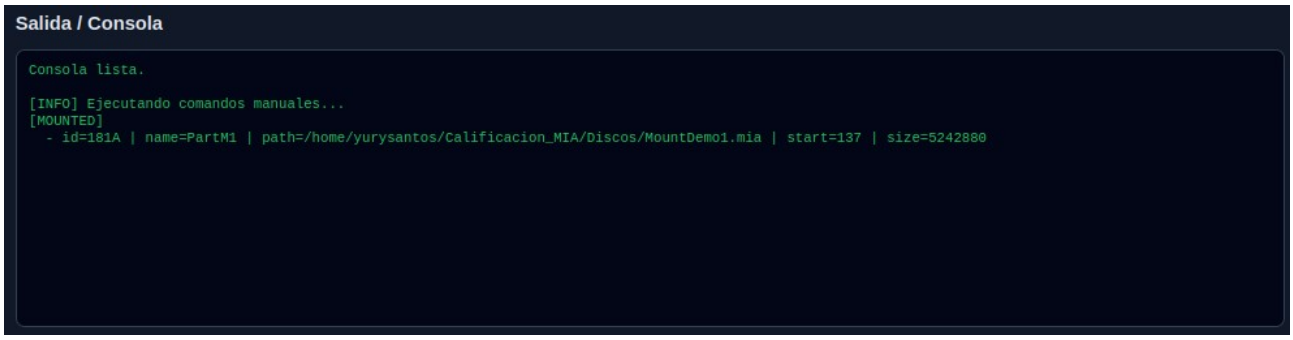
Propósito

Permite visualizar las particiones activas, sus IDs de montaje y datos asociados.

Parámetros principales

No requiere parámetros.

Ejemplo



Efecto en las estructuras internas

No modifica estructuras del sistema.
Solo consulta la tabla de montajes en memoria y presenta:

- ID de montaje
- nombre de la partición
- ruta del disco
- posición inicial
- tamaño

Comandos del sistema de archivos

MKFS

El comando `mkfs` formatea una partición montada en EXT2.

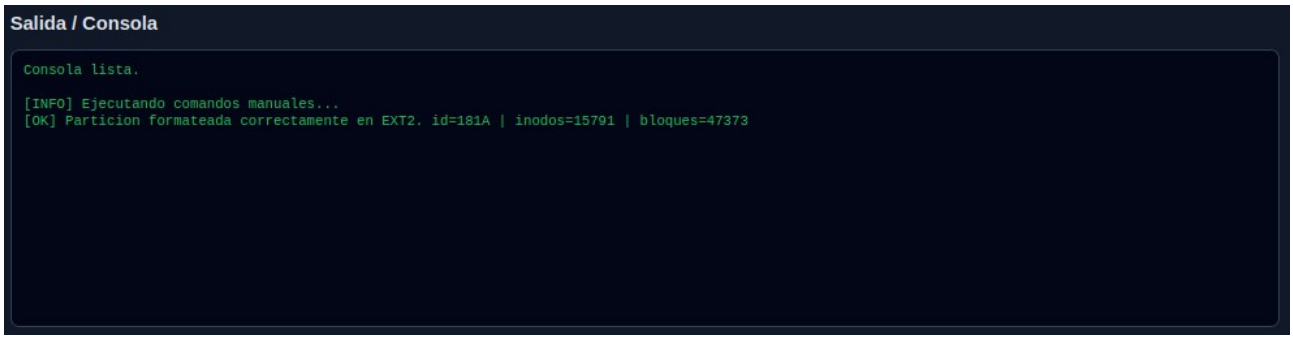
Propósito

Inicializa la partición seleccionada para que funcione como un sistema de archivos EXT2 simulado.

Parámetros principales

- `-id=` ID de la partición montada
- `-type=` tipo de formateo, cuando aplica

Ejemplo



Efecto en las estructuras internas

Este comando crea y/o inicializa:

- SuperBloque
- bitmap de inodos
- bitmap de bloques
- tabla de inodos
- bloques de datos
- carpeta raíz
- archivo especial `users.txt`

También:

- marca como ocupados los primeros inodos y bloques utilizados
- deja listo el sistema para aceptar sesiones, carpetas, archivos y reportes

CAT

El comando `cat` muestra el contenido de uno o varios archivos.

Propósito

Permite leer el contenido textual de archivos almacenados dentro del sistema de archivos EXT2 simulado.

Parámetros principales

- `-file1=`
- `-file2=`
- y así sucesivamente, según la implementación

Ejemplo

Efecto en las estructuras internas

No modifica estructuras.
El sistema:

- recorre la ruta indicada
- localiza el inodo del archivo
- obtiene los bloques asociados
- concatena el contenido
- lo imprime en la salida

Estructuras involucradas

- bloques de carpeta para localizar la ruta
- inodo del archivo
- bloques de archivo con el contenido real

Comandos de sesión, usuarios y grupos

LOGIN

El comando `login` inicia sesión dentro de una partición formateada.

Propósito

Permite autenticar un usuario registrado en `users.txt` y activar una sesión para operar el sistema.

Parámetros principales

- `-user=`
- `-pass=`
- `-id=`

Ejemplo

Efecto en las estructuras internas

No modifica el disco directamente, pero sí actualiza la sesión en memoria:

- usuario actual
- grupo actual
- partición activa
- permisos operativos

Además, el sistema lee `users.txt` para validar credenciales.

LOGOUT

El comando `logout` finaliza la sesión activa.

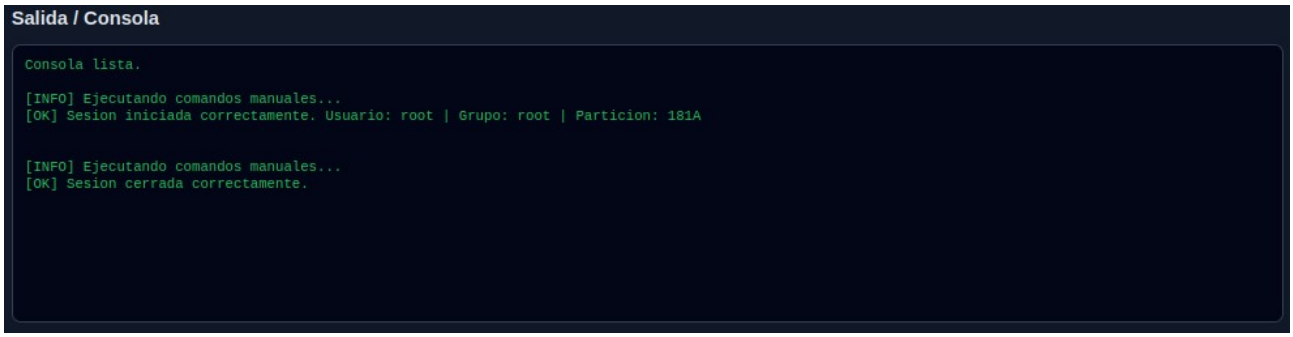
Propósito

Cerrar la sesión actual y limpiar el contexto de usuario.

Parámetros principales

No requiere parámetros.

Ejemplo



Efecto en las estructuras internas

No modifica estructuras del disco.
Limpia el estado de sesión almacenado en memoria.

MKGRP

El comando `mkgrp` crea un grupo nuevo dentro de `users.txt`.

Propósito

Permitir la administración de grupos del sistema.

Parámetros principales

- `-name=`

Ejemplo

Efecto en las estructuras internas

- se localiza el archivo `users.txt`
- se agrega una nueva línea con el grupo
- se actualizan los bloques de archivo correspondientes
- si el contenido crece, pueden utilizarse bloques adicionales

RMGRP

El comando `rmgrp` elimina lógicamente un grupo.

Propósito

Permitir que un grupo ya no sea considerado activo sin borrar físicamente la línea del archivo.

Parámetros principales

- `-name=`

Ejemplo

Efecto en las estructuras internas

- se localiza el grupo dentro de `users.txt`
- se marca como eliminado de forma lógica
- el contenido del archivo se reescribe con el valor de eliminación correspondiente

MKUSR

El comando `mkusr` crea un nuevo usuario.

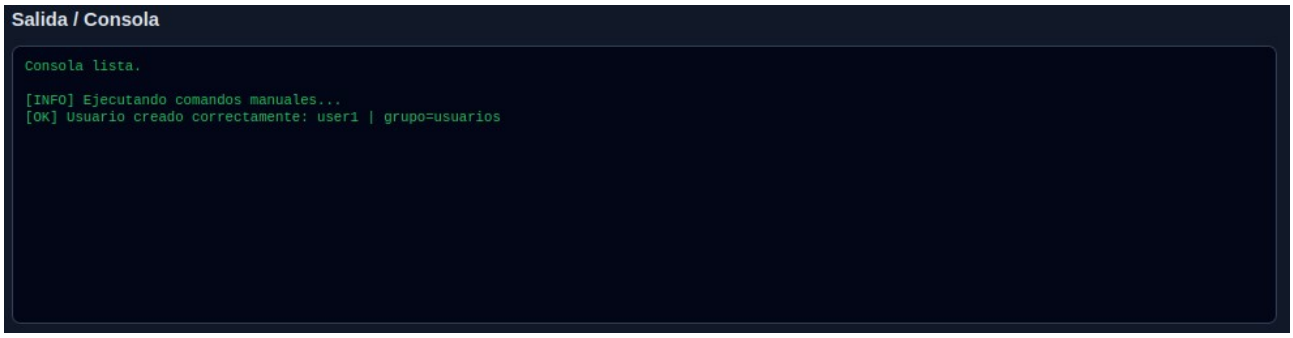
Propósito

Registrar usuarios válidos para autenticación y control de acceso.

Parámetros principales

- `-user=`
- `-pass=`
- `-grp=`

Ejemplo



Efecto en las estructuras internas

- verifica que el grupo exista
- verifica que el usuario no exista previamente
- agrega una nueva línea en `users.txt`
- actualiza bloques del archivo según el tamaño del contenido

RMUSR

El comando `rmusr` elimina lógicamente un usuario.

Propósito

Desactivar un usuario sin borrar físicamente su registro del archivo.

Parámetros principales

- `-user=`

Ejemplo

Efecto en las estructuras internas

- se localiza el usuario en `users.txt`
- se marca como eliminado lógicamente
- se reescribe el contenido del archivo

CHGRP

El comando `chgrp` cambia el grupo asociado a un usuario existente.

Propósito

Actualizar la afiliación de un usuario a un grupo válido del sistema.

Parámetros principales

- `-user=`
- `-grp=`

Ejemplo

Efecto en las estructuras internas

- se valida que el usuario exista
- se valida que el grupo destino exista
- se modifica la línea correspondiente en `users.txt`
- se actualizan los bloques de archivo según sea necesario

Comandos de carpetas y archivos

MKDIR

El comando `mkdir` crea carpetas dentro del sistema de archivos.

Propósito

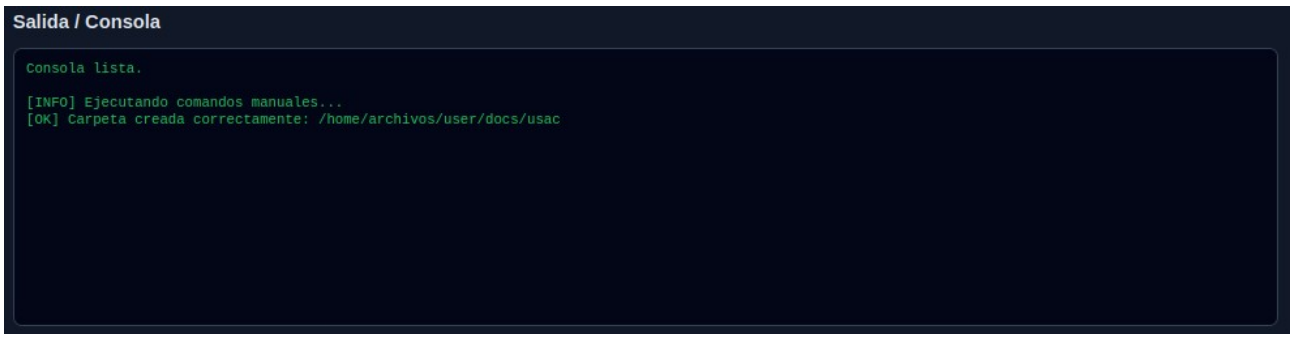
Permitir la creación de directorios, con o sin creación recursiva de carpetas padre.

Parámetros principales

- `-path=`
- `-p` para creación recursiva

Ejemplo simple

Ejemplo recursivo



Efecto en las estructuras internas

Al crear una carpeta, el sistema:

- recorre la ruta padre
- reserva un nuevo inodo de tipo carpeta
- reserva un nuevo bloque de carpeta
- agrega la entrada correspondiente en la carpeta padre
- actualiza bitmaps de inodos y bloques
- actualiza contadores del SuperBloque

Cuando se usa `-p`, el sistema crea también las carpetas intermedias que no existan.

MKFILE

El comando `mkfile` crea archivos dentro del sistema.

Propósito

Permitir la creación de archivos vacíos, con contenido generado por tamaño, con contenido externo o de forma recursiva.

Parámetros principales

- `-path=`
- `-size=`
- `-cont=`
- `-r` para creación recursiva

Ejemplo con tamaño

Ejemplo con contenido externo

Entrada de comandos

```
mkfile -path=/home/archivos/user/docs/Tarea3.txt -size=10 -cont=/home/yurysantos/Calificacion_MIA/CONT/NAME.txt
```

Ejecutar Limpiar

Salida / Consola

```
Consola lista.

[INFO] Ejecutando comandos manuales...
[OK] Archivo creado correctamente: /home/archivos/user/docs/Tarea3.txt
```

Ejemplo recursivo

Entrada de comandos

```
mkfile -r -path=/home/archivos/user/docs/usac/archivos/proyectos/fase1/entrada.txt
```

Ejecutar Limpiar

Salida / Consola

```
Consola lista.

[INFO] Ejecutando comandos manuales...
[OK] Archivo creado correctamente: /home/archivos/user/docs/usac/archivos/proyectos/fase1/entrada.txt
```

Efecto en las estructuras internas

Al crear un archivo, el sistema:

- localiza o crea la ruta padre
- reserva un nuevo inodo de archivo
- asigna uno o más bloques de archivo
- escribe contenido según el tamaño o archivo externo
- actualiza bitmaps de inodos y bloques
- registra la entrada del archivo en el bloque de carpeta correspondiente
- actualiza el SuperBloque

Casos de uso soportados

Archivo por tamaño

Se genera contenido automático para llenar el tamaño solicitado.

Archivo con contenido externo

Se lee un archivo del sistema operativo y su contenido se copia dentro del sistema EXT2 simulado.

Archivo recursivo

Si se usa `-r`, el sistema crea las carpetas faltantes antes de generar el archivo.

Comando de reportes

REP

El comando `rep` genera reportes visuales o de texto sobre estructuras internas del sistema.

Propósito

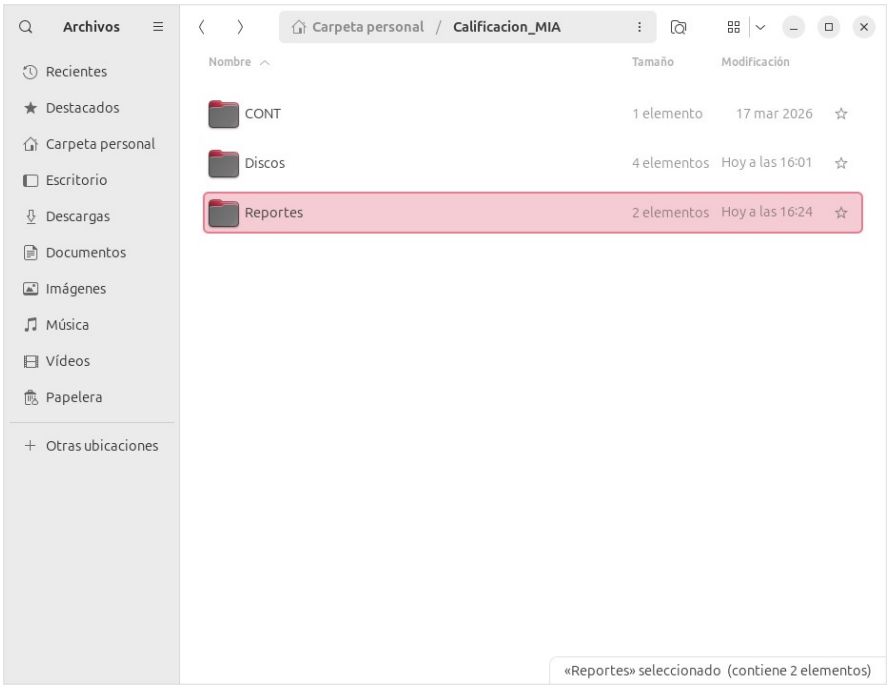
Permitir la inspección técnica del disco, particiones y sistema de archivos mediante salidas gráficas y reportes estructurados.

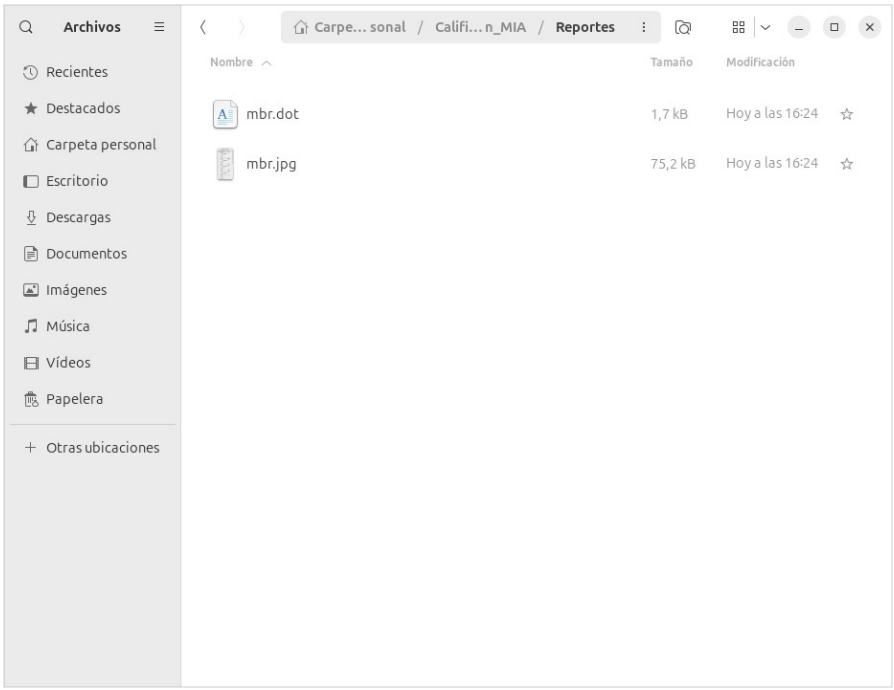
Parámetros principales

- `-id=`
- `-path=`
- `-name=`
- `-ruta=` para reportes que requieren una ruta interna específica

Ejemplo de reporte MBR

Ejemplo de reporte FILE





REPORTE DE MBR	
mbr_tamano	20971520
mbr_fecha_creacion	2026-04-02 15:53:34
mbr_disk_signature	862895609
disk_fit	f
PARTICION 1	
part_status	1
part_type	p
part_fit	b
part_start	137
part_size	5242880
part_name	PartM1
PARTICION 2	
part_status	1
part_type	p
part_fit	b
part_start	5243017
part_size	5242880
part_name	PartM2
PARTICION 3	
part_status	1
part_type	p
part_fit	b
part_start	10485897
part_size	3145728
part_name	PartM3
PARTICION 4	
part_status	0
part_type	p
part_fit	f
part_start	-1
part_size	0
part_name	

Reportes implementados

- `mbr`
- `disk`
- `inode`
- `block`
- `bm_inode`
- `bm_block`
- `sb`
- `file`
- `ls`
- `tree`

Efecto en las estructuras internas

Los reportes no modifican el sistema.
Su función es leer estructuras y representarlas.

Cada reporte utiliza distintas estructuras:

MBR
Lee el MBR y las entradas de partición.

DISK
Interpreta la distribución física del disco y las regiones libres u ocupadas.

INODE
Lee la tabla de inodos y muestra sus campos.

BLOCK
Lee los bloques de datos del sistema.

bm_inode
Representa el bitmap de inodos.

bm_block
Representa el bitmap de bloques.

SB
Lee el SuperBloque.

TREE
Recorre la relación entre inodos y bloques para mostrar la estructura jerárquica.

FILE
Ubica una ruta específica y muestra el contenido del archivo.

LS
Ubica una carpeta y presenta los elementos contenidos en ella, junto con metadatos.

Validaciones generales de los comandos

Todos los comandos pasan por una fase de validación antes de ejecutarse. Entre las validaciones implementadas se encuentran:

- existencia de parámetros obligatorios
- rechazo de parámetros inválidos
- existencia de archivos o rutas del sistema operativo
- control de sesión activa
- verificación de permisos
- nombres duplicados
- longitudes máximas permitidas
- restricciones de teoría de particiones
- verificación de espacio disponible
- validación de rutas internas del sistema de archivos

Estas validaciones permiten que el sistema no solo ejecute casos correctos, sino que también responda apropiadamente a errores esperados.

Relación entre comandos y módulos internos

Cada comando es procesado por el `Analyzer` y derivado al módulo correspondiente.

Comandos asociados a `DiskManagement`

- `mkdisk`
- `rmdisk`
- `fdisk`

Comandos asociados a `MountManager`

- `mount`
- `mounted`

Comandos asociados a `FileSystemManager`

- `mkfs`
- `mkdir`
- `mkfile`
- `cat`

Comandos asociados a `SessionManager`

- `login`
- `logout`
- `mkgrp`
- `rmgrp`
- `mkusr`
- `rmusr`
- `chgrp`

Comandos asociados a `ReportManager`

- `rep`

Proceso de compilación, ejecución e integración frontend-backend

El proyecto ExtreamFS fue desarrollado con una arquitectura separada en dos capas principales: un backend en C++ y un frontend en Angular. Para que el sistema funcione correctamente, ambas capas deben compilarse y ejecutarse de forma independiente, manteniendo una comunicación local a través de HTTP.

Esta sección describe el proceso técnico de compilación del backend, la ejecución del frontend y la integración entre ambas partes.

Compilación del backend

El backend fue implementado en C++ y se compila mediante CMake. La finalidad de esta capa es procesar toda la lógica del sistema, incluyendo:

- análisis de comandos
- administración de discos y particiones
- formateo EXT2
- administración de usuarios y grupos
- manejo de carpetas y archivos
- generación de reportes
- exposición de endpoints HTTP para el frontend

La compilación del backend se realiza desde la carpeta `backend`, donde se encuentra el archivo `CMakeLists.txt`.

Comando de compilación

```
yurysantos@yurysantos-Alienware-x17-R1:~/Escritorio/MIA_1S2026_P1_202113318/ExtreamFS/
backend$ cmake --build build -j
[100%] Built target extreamfs
```

Ejecución del backend

Para que el frontend pueda comunicarse con el sistema, el backend debe levantarse en modo API. En este modo, el ejecutable inicia un servidor HTTP local que escucha solicitudes provenientes de la aplicación web.

Comando de ejecución en modo API

```
yurysantos@yurysantos-Alienware-x17-R1:~/Escritorio/MIA_1S2026_P1_202113318/ExtreamFS/
backend$ ./build/extreamfs --api
[INFO] Backend HTTP escuchando en http://0.0.0.0:8080
```

Esto indica que el servidor HTTP quedó activo en el puerto `8080`.

Función del backend en modo API

En este modo, el backend se encarga de:

- recibir peticiones del frontend
- interpretar el contenido enviado
- procesar comandos o scripts completos
- devolver respuestas en formato JSON
- conservar en memoria estado temporal, como particiones montadas y sesión activa

Compilación y ejecución del frontend

El frontend fue desarrollado con Angular y tiene la función de servir como interfaz visual para el usuario. A través de esta aplicación web se pueden:

- escribir comandos manualmente
- cargar archivos `.smia`
- ejecutar scripts completos
- visualizar comentarios, errores y resultados
- comprobar si el backend está conectado

La aplicación Angular se ejecuta desde la carpeta del proyecto frontend.

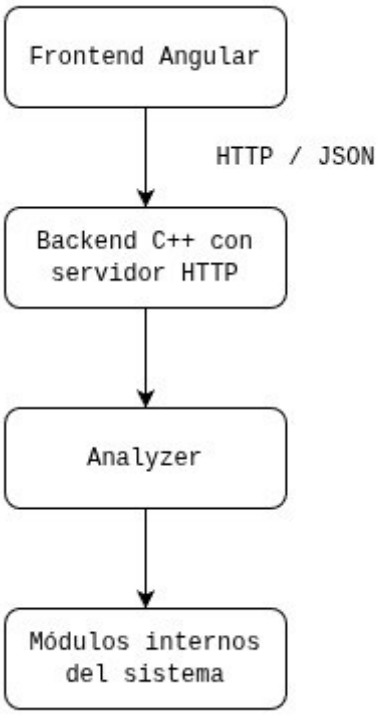
Comando para levantar el frontend

```
yurysantos@yurysantos-Alienware-x17-R1:~/Escritorio/MIA_1S2026_P1_202113318/ExtreamFS
/frontend/extreamfs-frontend$ ng serve -o
Browser bundles
Initial chunk files     | Names               |  Raw size
polyfills.js            | polyfills           |  88.09 kB |
main.js                 | main                |  12.30 kB |
styles.css              | styles              |  95 bytes |

                        | Initial total       | 100.48 kB


Server bundles
Initial chunk files     | Names               |  Raw size
chunk-PZGT2MSS.mjs      | -                   |   1.67 MB |
polyfills.server.mjs    | polyfills.server    | 559.49 kB |
main.server.mjs         | main.server         | 398.80 kB |
chunk-VPSODEBW.mjs      | -                   |   2.51 kB |
render-utils.server.mjs | render-utils.server | 423 bytes |

Lazy chunk files        | Names               |  Raw size
chunk-P3UOPGAK.mjs      | xhr2                |  39.09 kB |

Application bundle generation complete. [2.853 seconds]

Watch mode enabled. Watching for file changes...
  → Local:   http://localhost:4200/
  → press h + enter to show help
NG02801: Angular detected that `HttpClient` is not configured to use `fetch` APIs. It
's strongly recommended to enable `fetch` for applications that use Server-Side Rende
ring for better performance and compatibility. To enable `fetch`, add the `withFetch(
)` to the `provideHttpClient()` call at the root of the application.
```

Integración entre frontend y backend

La integración entre ambas capas se realiza mediante una API HTTP local. Esta decisión permitió reutilizar toda la lógica ya existente del backend en C++ y, al mismo tiempo, exponerla a una interfaz moderna y fácil de usar.

Esquema general de integración



El frontend no ejecuta lógica de negocio directamente. Su responsabilidad es:

- capturar la interacción del usuario
- preparar el contenido que será enviado
- consumir la respuesta del backend
- mostrar el resultado en la consola web

Toda la lógica de procesamiento permanece centralizada en el backend.

Papel del Analyzer en la integración

Uno de los puntos técnicos más importantes del proyecto fue la reutilización del `Analyzer` como núcleo de procesamiento tanto en modo CLI como en modo web.

Originalmente, el sistema funcionaba desde consola interactiva. Para integrarlo con el frontend, se adaptó el analizador para soportar también la ejecución de bloques de texto completos.

Método agregado

Se incorporó un método público capaz de:

- recibir un script completo
- separar el contenido por líneas
- conservar comentarios
- ignorar líneas vacías
- ejecutar cada instrucción
- capturar toda la salida
- devolver el resultado como `string`

Gracias a esto, el frontend puede enviar un archivo completo `.smia` o un bloque manual de instrucciones y el backend lo procesa con la misma lógica interna que ya utilizaba en la consola.

Flujo de ejecución de un comando manual

Cuando el usuario escribe un comando en la sección Entrada de comandos y presiona Ejecutar, ocurre lo siguiente:

1. El frontend toma el contenido del área de texto.
2. El contenido se envía por HTTP al endpoint `POST /api/execute`.
3. El backend recibe la solicitud.
4. `HttpServer` extrae el bloque de texto enviado.
5. El texto se entrega al `Analyzer`.
6. El `Analyzer` interpreta línea por línea.
7. El comando es dirigido al módulo correspondiente.
8. El módulo ejecuta la operación sobre el disco o sistema de archivos.
9. La salida generada se captura.
10. El backend responde al frontend con un JSON.
11. El frontend imprime el resultado en la sección **Salida / Consola**.

Flujo de ejecución de un archivo `.smia`

Cuando el usuario carga un archivo `.smia` desde la sección destinada a scripts, el frontend realiza el siguiente proceso:

1. El usuario selecciona el archivo desde el botón Examinar.
2. El frontend lee el archivo con `FileReader`.
3. El contenido se muestra en un área de texto de solo lectura.
4. Al presionar Ejecutar archivo, el contenido completo se envía al backend.
5. El backend lo procesa como un bloque de instrucciones.
6. Los comentarios, mensajes de error y resultados aparecen en la consola web.

Este flujo fue verificado exitosamente con el archivo oficial adaptado del proyecto, demostrando que la interfaz ya no solo carga el archivo, sino que también lo ejecuta correctamente de punta a punta.

Dependencias y herramientas utilizadas

En el backend se utilizaron:

- C++
- CMake
- Graphviz
- librería HTTP header-only para el servidor local

En el frontend se utilizaron:

- Angular
- TypeScript
- HTML
- CSS
- `HttpClient` para consumo del API

Estas herramientas permiten un entorno modular, mantenible y suficientemente flexible para el desarrollo del proyecto.

Consideraciones de ejecución

Estado en memoria del backend

El backend conserva información en memoria mientras el proceso sigue activo, especialmente:

- particiones montadas
- sesión actual
- estado temporal necesario para la ejecución

Por ello, para corridas oficiales de prueba donde se esperan IDs exactos como `181A`, `182A`, `181B`, `182B` y `181C`, se recomienda reiniciar el backend antes de ejecutar el script completo.

Limpieza previa recomendada

Antes de ejecutar un archivo oficial de prueba, se recomienda:

- detener backend y frontend
- borrar discos de prueba viejos
- limpiar la carpeta de reportes
- volver a iniciar el backend en modo API
- volver a iniciar el frontend

Esto garantiza una ejecución limpia y resultados consistentes.

Ventajas de la integración implementada

La integración frontend-backend adoptada en el proyecto ofrece varias ventajas:

- permite mantener toda la lógica en C++
- evita duplicar lógica entre consola y web
- facilita pruebas con comandos manuales y archivos completos
- mejora la experiencia de usuario respecto al modo CLI puro
- permite visualizar el estado del sistema desde una interfaz gráfica
- hace posible una demostración más clara durante la calificación

Resultado de la integración

La integración desarrollada permitió lograr un sistema funcional donde:

- el backend se mantiene como núcleo de procesamiento
- el frontend actúa como cliente de la API
- los comandos se ejecutan correctamente desde la web
- los archivos `.smia` se cargan y se ejecutan desde la interfaz
- la salida del sistema se muestra en tiempo real en la consola del frontend

Esto permitió cumplir con la parte del proyecto relacionada con la aplicación web y con la comunicación correcta entre frontend y backend.

Generación de reportes con Graphviz

El proyecto ExtreamFS implementa la generación de reportes gráficos y de texto como parte fundamental del análisis y verificación interna del sistema de archivos. Para ello se utilizó Graphviz, una herramienta especializada en la representación de estructuras mediante descripciones en formato DOT.

El uso de Graphviz permitió convertir la información binaria del disco y del sistema de archivos en diagramas legibles y visualmente comprensibles, facilitando tanto la depuración del proyecto como la validación del funcionamiento interno de las estructuras implementadas.

Propósito de los reportes

Los reportes tienen como objetivo mostrar de manera clara la información interna del sistema, incluyendo:

- metadatos del disco
- distribución de particiones
- información del SuperBloque
- contenido de inodos
- contenido de bloques
- bitmaps
- estructura jerárquica del sistema de archivos
- contenido de archivos específicos
- listado de carpetas

Esto permite observar visualmente cómo está organizado el disco virtual y cómo se distribuyen los datos dentro de la partición formateada.

Herramienta utilizada

Para la generación de reportes se utilizó Graphviz, específicamente mediante archivos de descripción en formato DOT, los cuales luego se convierten a formatos de salida como:

- `.jpg`
- `.png`

En el caso de los bitmaps, algunos reportes se generan como archivos de texto:

- `.txt`

Graphviz fue elegido porque permite:

- construir tablas estructuradas
- representar relaciones jerárquicas
- generar diagramas claros
- integrar fácilmente información dinámica generada desde C++

Módulo encargado de los reportes

La generación de reportes está centralizada en el módulo `ReportManager`.

Este componente es responsable de:

- recibir la solicitud del comando `rep`
- validar el tipo de reporte solicitado
- localizar la partición montada usando su ID
- leer las estructuras binarias necesarias
- construir el contenido DOT o de texto
- crear carpetas de salida si es necesario
- invocar Graphviz para generar el archivo final

Dentro del backend, este módulo trabaja en coordinación con:

- `MountManager`, para resolver el ID de la partición montada
- `FileSystemManager`, para acceder a estructuras del sistema de archivos
- lectura directa del archivo binario `.mia`

Flujo de generación de reportes

El flujo general para generar un reporte es el siguiente:

1. El usuario ejecuta el comando `rep`.
2. El `Analyzer` identifica el comando y sus parámetros.
3. El comando es enviado a `ReportManager`.
4. El sistema valida:
    - el ID de partición montada
    - la ruta de salida
    - el nombre del reporte
    - la ruta interna, cuando aplica
5. Se abre el disco binario correspondiente.
6. Se leen las estructuras necesarias según el reporte solicitado.
7. Se construye el contenido del reporte:
    - en formato DOT, si es gráfico
    - en texto, si es bitmap
8. Se escribe un archivo temporal o final.
9. Se invoca Graphviz para generar la imagen final.
10. El sistema devuelve un mensaje de éxito o error.

Estructura general del comando `rep`

El comando `rep` permite solicitar distintos tipos de reportes utilizando parámetros comunes.

Parámetros principales

- `-id=` identifica la partición montada
- `-path=` indica la ruta de salida del archivo
- `-name=` indica el nombre del reporte
- `-ruta=` se utiliza en reportes que requieren una ruta interna específica del sistema de archivos

Reportes implementados

El sistema implementa los siguientes reportes:

- `mbr`
- `disk`
- `inode`
- `block`
- `bm_inode`
- `bm_block`
- `sb`
- `tree`
- `file`
- `ls`

Cada reporte utiliza distintas estructuras y técnicas de representación.

Reporte MBR

El reporte `mbr` muestra la información del Master Boot Record y de las particiones registradas dentro del disco.

Información mostrada

- tamaño del disco
- fecha de creación
- firma del disco
- ajuste general
- información de las particiones
    - estado
    - tipo
    - ajuste
    - inicio
    - tamaño
    - nombre

Tipo de representación

Se genera como una tabla mediante Graphviz, donde cada fila corresponde a un campo o una partición.

Utilidad

Permite verificar:

- que el disco fue creado correctamente
- que las particiones están registradas en el MBR
- que los tamaños e inicios son coherentes

REPORTE DE MBR	
mbr_tamano	52428800
mbr_fecha_creacion	2026-04-02 17:09:50
mbr_disk_signature	328428832
disk_fit	f
PARTICION 1	
part_status	1
part_type	p
part_fit	b
part_start	137
part_size	10485760
part_name	Part11
PARTICION 2	
part_status	1
part_type	p
part_fit	b
part_start	10485897
part_size	10485760
part_name	Part12
PARTICION 3	
part_status	1
part_type	p
part_fit	b
part_start	20971657
part_size	10485760
part_name	Part13
PARTICION 4	
part_status	1
part_type	p
part_fit	b
part_start	31457417
part_size	10485760
part_name	Part14

Reporte DISK

El reporte `disk` representa la distribución física del disco.

Información mostrada

- MBR
- particiones primarias
- particiones extendidas
- particiones lógicas
- espacios libres

Tipo de representación

Se genera gráficamente como una barra o tabla horizontal donde cada segmento representa una región del disco.

Utilidad

Permite verificar:

- distribución del espacio
- porcentaje ocupado y libre
- ubicación relativa de particiones
- presencia de regiones extendidas y lógicas

REPORTE DISK					
MBR 0.000261307%	**PRIMARIA** Part11 20%	**PRIMARIA** Part12 20%	**PRIMARIA** Part13 20%	**PRIMARIA** Part14 20%	**LIBRE** 19.9997%

Reporte INODE

El reporte `inode` muestra los inodos ocupados dentro del sistema de archivos.

Información mostrada

Por cada inodo se representan campos como:

- UID
- GID
- tamaño
- fechas
- tipo
- permisos
- apuntadores

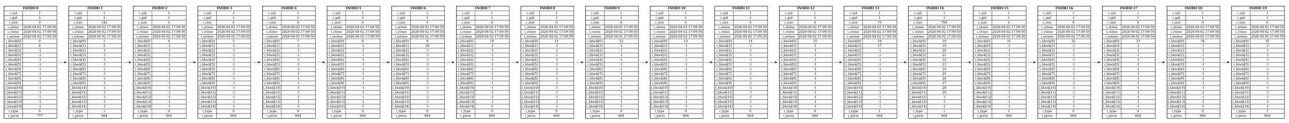
Tipo de representación

Se genera mediante tablas Graphviz, una por cada inodo utilizado.

Utilidad

Permite verificar:

- qué inodos están activos
- cómo están organizados sus apuntadores
- qué tipo de objeto representa cada inodo



INODO 0	
i_uid	1
i_gid	1
i_size	0
i_atime	2026-04-02 17:09:50
i_ctime	2026-04-02 17:09:50
i_mtime	2026-04-02 17:09:50
i_block[0]	0
i_block[1]	6
i_block[2]	-1
i_block[3]	-1
i_block[4]	-1
i_block[5]	-1
i_block[6]	-1
i_block[7]	-1
i_block[8]	-1
i_block[9]	-1
i_block[10]	-1
i_block[11]	-1
i_block[12]	-1
i_block[13]	-1
i_block[14]	-1
i_type	0
i_perm	777

INODO 1	
i_uid	1
i_gid	1
i_size	142
i_atime	2026-04-02 17:09:50
i_ctime	2026-04-02 17:09:50
i_mtime	2026-04-02 17:09:50
i_block[0]	1
i_block[1]	2
i_block[2]	3
i_block[3]	-1
i_block[4]	-1
i_block[5]	-1
i_block[6]	-1
i_block[7]	-1
i_block[8]	-1
i_block[9]	-1
i_block[10]	-1
i_block[11]	-1
i_block[12]	-1
i_block[13]	-1
i_block[14]	-1
i_type	1
i_perm	664

Reporte BLOCK

El reporte `block` representa los bloques de datos utilizados por el sistema.

Información mostrada

Dependiendo del tipo de bloque, puede mostrar:

- entradas de carpetas
- contenido textual de archivos

Tipo de representación

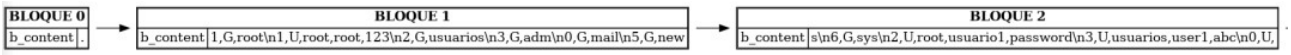
Cada bloque se representa como una tabla o recuadro independiente en Graphviz.

Utilidad

Permite verificar:

- contenido de directorios
- contenido de archivos
- relación entre inodos y bloques
- ocupación real del sistema de archivos

BLOQUE 2	
b_content	s\n6,G,sys\n2,U,root,usuario1,password\n3,U,usuarios,user1,abc\n0,U,



Reporte bm_inode

El reporte `bm_inode` muestra el bitmap de inodos.

Información mostrada

Representa qué inodos están:

- libres
- ocupados

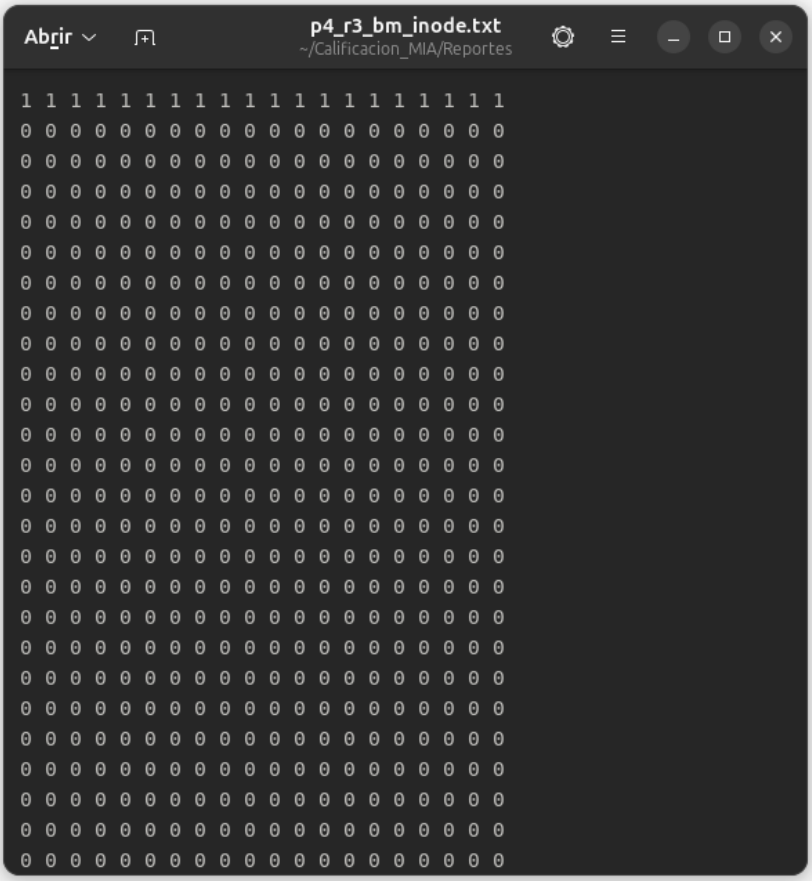
Tipo de representación

Este reporte se genera como archivo de texto.

Utilidad

Permite verificar:

- cuántos inodos están siendo utilizados
- si la asignación de inodos coincide con las operaciones realizadas



Reporte bm_block

El reporte `bm_block` muestra el bitmap de bloques.

Información mostrada

Indica qué bloques están:

- libres
- ocupados

Tipo de representación

Se genera como archivo de texto.

Utilidad

Permite verificar:

- consumo de bloques
- crecimiento del sistema de archivos
- coherencia entre archivos/carpetas creados y bloques ocupados

Reporte SB

El reporte `sb` representa el **SuperBloque** de una partición formateada.

Información mostrada

Incluye campos como:

- tipo de sistema de archivos
- número total de inodos
- número total de bloques
- inodos libres
- bloques libres
- fechas
- contador de montajes
- tamaño del inodo
- tamaño del bloque
- posiciones de inicio de bitmaps, inodos y bloques

Tipo de representación

Se genera como tabla Graphviz.

Utilidad

Permite verificar:

- que `mkfs` se ejecutó correctamente
- ubicación de estructuras internas
- estado general del sistema de archivos

SUPER BLOQUE	
s_filesystem_type	2
s_inodes_count	31583
s_blocks_count	94749
s_free_blocks_count	94716
s_free_inodes_count	31566
s_mtime	2026-04-02 17:09:50
s_umtime	2026-04-02 17:09:50
s_mnt_count	1
s_magic	61267
s_inode_size	136
s_block_size	64
s_first_ino	20
s_first_blo	36
s_bm_inode_start	237
s_bm_block_start	31820
s_inode_start	126569
s_block_start	4421857

Reporte TREE

El reporte `tree` representa la estructura jerárquica completa del sistema de archivos.

Información mostrada

- inodos
- bloques
- relaciones entre ellos
- rutas jerárquicas de carpetas y archivos

Tipo de representación

Se genera como un grafo dirigido en Graphviz, donde:

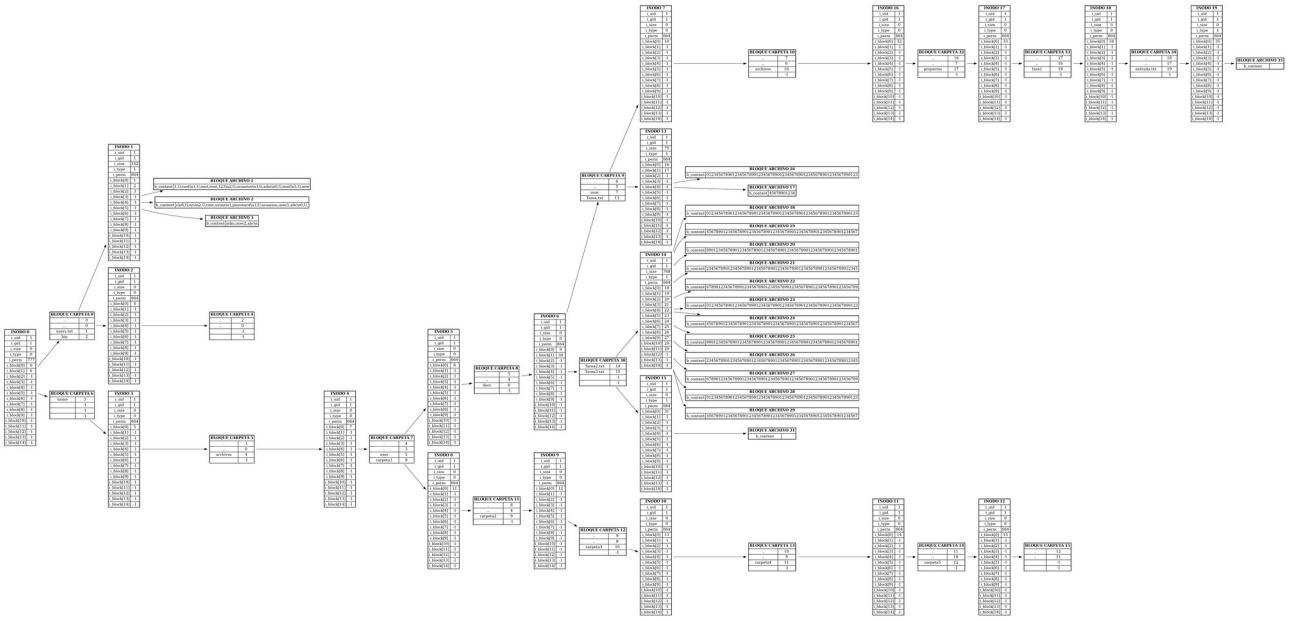
- los nodos representan inodos y bloques
- las flechas representan referencias o apuntadores

Utilidad

Permite observar visualmente:

- la estructura de carpetas
- la relación entre inodos y bloques
- cómo el sistema almacena archivos y directorios

Este es uno de los reportes más útiles para explicar el funcionamiento interno del proyecto.



INODO 1	
i_uid	1
i_gid	1
i_size	142
i_type	1
i_perm	664
i_block[0]	1
i_block[1]	2
i_block[2]	3
i_block[3]	-1
i_block[4]	-1
i_block[5]	-1
i_block[6]	-1
i_block[7]	-1
i_block[8]	-1
i_block[9]	-1
i_block[10]	-1
i_block[11]	-1
i_block[12]	-1
i_block[13]	-1
i_block[14]	-1

BLOQUE ARCHIVO 1	
b_content	1,G,root\n1,U,root,root,123\n2,G,usuarios\n3,G,adm\n0,G,mail\n5,G,new

BLOQUE ARCHIVO 2	
b_content	s\n6,G,sys\n2,U,root,usuario1,password\n3,U,usuarios,user1,abc\n0,U,

BLOQUE ARCHIVO 3	
b_content	adm,user2,abc\n

INODO 13	
i_uid	1
i_gid	1
i_size	75
i_type	1
i_perm	664
i_block[0]	16
i_block[1]	17
i_block[2]	-1
i_block[3]	-1
i_block[4]	-1
i_block[5]	-1
i_block[6]	-1
i_block[7]	-1
i_block[8]	-1
i_block[9]	-1
i_block[10]	-1
i_block[11]	-1
i_block[12]	-1
i_block[13]	-1
i_block[14]	-1

BLOQUE ARCHIVO 16	
b_content	0123456789012345678901234567890123456789012345678901234567890123

BLOQUE ARCHIVO 17	
b_content	45678901234

BLOQUE ARCHIVO 18	
b_content	0123456789012345678901234567890123456789012345678901234567890123

BLOQUE ARCHIVO 19	
b_content	4567890123456789012345678901234567890123456789012345678901234567

BLOQUE ARCHIVO 20	
b_content	8901234567890123456789012345678901234567890123456789012345678901

BLOQUE ARCHIVO 21	
b_content	2345678901234567890123456789012345678901234567890123456789012345

INODO 14	
i_uid	1
i_gid	1
i_size	768
i_type	1

Reporte FILE

El reporte `file` muestra el contenido de un archivo específico dentro del sistema de archivos.

Información mostrada

- ruta interna del archivo
- contenido textual del archivo solicitado

Parámetro adicional requerido

- `-ruta=`

Ejemplo

REPORTE FILE
Ruta: /home/archivos/user/docs/Tarea2.txt
1234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901234567

Tipo de representación

Se genera como tabla Graphviz con el contenido del archivo.

Utilidad

Permite verificar:

- que el archivo existe
- que la ruta interna fue localizada correctamente
- que el contenido almacenado coincide con lo esperado

Tipo de representación

Se genera como tabla Graphviz con el contenido del archivo.

Utilidad

Permite verificar:

- que el archivo existe
- que la ruta interna fue localizada correctamente
- que el contenido almacenado coincide con lo esperado

REPORTE LS							
Ruta: /home/archivos/user/docs							
Permisos	**Owner**	**Grupo**	**Size (bytes)**	**Fecha**	**Hora**	**Tipo**	**Name**
664	1	1	0	2026-04-02	17:09:50	Carpeta	.
664	1	1	0	2026-04-02	17:09:50	Carpeta	..
664	1	1	0	2026-04-02	17:09:50	Carpeta	usac
664	1	1	75	2026-04-02	17:09:50	Archivo	Tarea.txt
664	1	1	768	2026-04-02	17:09:50	Archivo	Tarea2.txt
664	1	1	0	2026-04-02	17:09:50	Archivo	Tarea3.txt

Tipo de representación

Se genera como una tabla con el contenido de la carpeta consultada.

Utilidad

Permite verificar:

- estructura interna de una carpeta
- archivos contenidos
- nombres y relaciones dentro del sistema

Construcción del contenido DOT

Para los reportes gráficos, el sistema construye dinámicamente texto en formato DOT. Este formato describe:

- nodos
- tablas
- etiquetas
- relaciones
- estilos básicos

Ejemplo conceptual de DOT

home > yurysantos > Calificacion_MIA > Reportes > p4_r7_ls.dot

```
digraph G {
  node [shape=plaintext];
  lsrep [label=<
  <table border='1' cellborder='1' cellspacing='0'>
    <tr><td colspan='8'><b>REPORTE LS</b></td></tr>
    <tr><td colspan='8'><b>Ruta:</b> /home/archivos/user/docs</td></tr>
    <tr><td><b>Permisos</b></td><td><b>Owner</b></td><td><b>Grupo</b></td><td><b>Size (bytes)</b></td><td><b>Fecha</b></td><td><b>
    <tr><td>664</td><td>1</td><td>1</td><td>0</td><td>2026-04-02</td><td>17:09:50</td><td>Carpeta</td><td>.</td></tr>
    <tr><td>664</td><td>1</td><td>1</td><td>0</td><td>2026-04-02</td><td>17:09:50</td><td>Carpeta</td><td>..</td></tr>
    <tr><td>664</td><td>1</td><td>1</td><td>0</td><td>2026-04-02</td><td>17:09:50</td><td>Carpeta</td><td>usac</td></tr>
    <tr><td>664</td><td>1</td><td>1</td><td>75</td><td>2026-04-02</td><td>17:09:50</td><td>Archivo</td><td>Tarea.txt</td></tr>
    <tr><td>664</td><td>1</td><td>1</td><td>768</td><td>2026-04-02</td><td>17:09:50</td><td>Archivo</td><td>Tarea2.txt</td></tr>
    <tr><td>664</td><td>1</td><td>1</td><td>0</td><td>2026-04-02</td><td>17:09:50</td><td>Archivo</td><td>Tarea3.txt</td></tr>
  </table>
  >];
}
```

Generación de la imagen final

Una vez construido el contenido DOT, el sistema:

- lo escribe en un archivo temporal o intermedio
- llama a Graphviz desde C++
- solicita la conversión al formato de salida deseado

Dependiendo del tipo de reporte, la salida puede generarse como:

- `.jpg`
- `.png`

En el caso de los bitmaps, el sistema escribe directamente un archivo `.txt` con la representación correspondiente.

Validaciones realizadas en los reportes

Antes de generar un reporte, el sistema verifica:

- que el ID de la partición exista y esté montado
- que el disco pueda abrirse correctamente
- que la partición contenga estructuras válidas
- que la ruta de salida sea válida
- que la carpeta de salida exista o pueda crearse
- que la ruta interna exista, si el reporte la requiere

Estas validaciones son importantes para evitar reportes inconsistentes o vacíos.

Relación entre reportes y estructuras internas

Cada tipo de reporte accede a una parte específica del sistema:

- `mbr` y `disk` se apoyan en estructuras del disco y particiones
- `sb`, `inode`, `block`, `bm_inode` y `bm_block` leen estructuras EXT2 internas
- `tree`, `file` y `ls` recorren el sistema de archivos mediante rutas, inodos y bloques

Esto convierte a los reportes en una herramienta útil no solo para presentación, sino también para validación técnica y depuración.

Uso práctico durante el desarrollo

Durante el desarrollo del proyecto, los reportes con Graphviz sirvieron para:

- comprobar que `fdisk` escribía correctamente las particiones
- verificar el contenido del MBR
- revisar la distribución de espacio en el disco
- confirmar la correcta inicialización de `mkfs`
- validar la ocupación de inodos y bloques
- comprobar la estructura jerárquica de carpetas y archivos
- visualizar contenido específico de archivos del sistema

Gracias a esto, los reportes se convirtieron en una herramienta esencial para confirmar que la simulación del sistema de archivos se comportaba conforme a la teoría esperada.

Integración con el frontend

Aunque la generación de reportes ocurre completamente en el backend, su invocación puede realizarse desde el frontend al ejecutar comandos `rep` dentro del área de entrada o mediante un archivo `.smia`.

El frontend no genera los reportes por sí mismo; únicamente envía el comando al backend y muestra el resultado en la consola, indicando si el archivo fue generado correctamente.

Esto permite que el usuario pueda:

- ordenar la generación de reportes desde la web
- revisar en consola si se creó correctamente
- abrir el archivo generado directamente desde el sistema operativo

Importancia técnica de Graphviz en el proyecto

El uso de Graphviz aporta valor técnico al proyecto porque:

- convierte estructuras binarias complejas en diagramas comprensibles
- mejora la presentación del funcionamiento interno del sistema
- facilita la verificación de estructuras implementadas
- ayuda a detectar errores estructurales durante el desarrollo
- cumple con el requerimiento del proyecto de usar reportes visuales

Conclusión

El proyecto ExtreamFS permitió desarrollar una solución funcional para la simulación de un sistema de archivos EXT2, integrando de forma correcta un backend en C++ con un frontend en Angular. Esta integración hizo posible combinar una interfaz web moderna con una lógica interna robusta capaz de administrar discos, particiones, usuarios, grupos, carpetas, archivos y reportes.

A nivel técnico, el sistema logró implementar satisfactoriamente las estructuras fundamentales requeridas por el proyecto, incluyendo el MBR, los EBR, el SuperBloque, los bitmaps, los inodos y los bloques, respetando la organización interna característica de un sistema de archivos tipo EXT2. Esto permitió que las operaciones realizadas por los comandos no solo produjeran resultados visibles para el usuario, sino que también modificaran coherentemente las estructuras binarias del disco virtual.

La organización modular del backend, dividida en componentes especializados como `Analyzer`, `DiskManagement`, `MountManager`, `FileSystemManager`, `SessionManager` y `ReportManager`, facilitó la separación de responsabilidades y mejoró la mantenibilidad del sistema. Además, la incorporación de una capa HTTP en C++ permitió reutilizar la lógica ya existente del proyecto y exponerla al frontend sin necesidad de duplicar funcionalidades.

El uso de Graphviz como herramienta para la generación de reportes aportó un valor importante al desarrollo, ya que permitió transformar estructuras internas complejas en representaciones gráficas comprensibles, útiles tanto para depuración como para validación y presentación del proyecto.

Las pruebas realizadas demostraron que el sistema responde correctamente tanto en escenarios exitosos como en casos de error controlado, validando parámetros, restricciones de teoría de particiones, control de sesión, existencia de rutas y consistencia de estructuras. De esta manera, el proyecto no solo cumple con los requerimientos funcionales, sino que también evidencia un comportamiento sólido y coherente ante distintas condiciones de ejecución.

En términos generales, ExtreamFS representa una implementación completa y bien estructurada de los conceptos estudiados en el curso, mostrando el uso práctico de archivos binarios, estructuras de bajo nivel, manejo de sistemas de archivos y desarrollo de aplicaciones integradas entre backend y frontend.